

Citizen Connect 360

Citizen Connect 360 is a comprehensive digital platform that bridges the gap between citizens and their government. It fosters transparency, accountability, and active engagement by providing a unified space for users to interact with government documents (such as parliamentary bills), report civic incidents, apply for bursaries, participate in polls, and file petitions. By leveraging Artificial Intelligence to analyze citizen input, the platform supports informed decision-making and drives positive change, ultimately leading to a more equitable society.

The platform enables citizens to see what their MPs are doing, propose bills, vote on issues, and participate in forums. MPs can communicate directly with constituents, publish their voting records, and manage proposed bills. Administrators manage the platform, users, and oversee the integrity of the data.

Key Features

- **User authentication & authorization** with three roles: *Constituent*, *MP (Member of Parliament)*, and *Admin*.
- **Bill tracker**: a system to track the progress of bills from proposal to enactment.
- **Forums/discussions**: spaces for public debate, optionally with live chat capabilities.
- **Polls/surveys**: for MPs to gauge constituent opinions on current issues.
- **Document repository**: centralized access to all official parliament documents.
- **Admin panel**: profiles for MPs showing their activity, voting history, and committee memberships.



Core Functional Requirements

1. User Authentication & Authorization

- Secure registration and login using email and password.
- Role-based access control (Constituent, MP, Admin).
- Password hashing with a strong algorithm (e.g., bcrypt or Argon2).
- Token-based authentication using JWT (access and refresh tokens).
- Session management with token expiration, refresh, and revocation.
- Password reset flow (request email → reset link → new password).
- Email verification for new accounts.
- Account deletion (with goodbye email).

2. Bill Tracking

- MPs can propose new bills with title, description, and supporting documents.
- Bills progress through statuses: *Draft, Introduced, In Committee, Passed, Rejected, Enacted*.
- Citizens can view all bills, filter by status, category, or MP sponsor.
- Each bill has a timeline of actions (e.g., committee reviews, votes).
- MPs can update the status of their own bills; admins can override.
- Citizens can comment on bills and express support/opposition.

3. Forums & Discussions

- Create topic threads under categories (e.g., Health, Education, Infrastructure).
- Users can post replies, quote others, and edit/delete their own posts.
- Moderation tools for admins and MPs (pin, lock, delete inappropriate content).
- Optional live chat using WebSockets for real-time discussion on a topic.

4. Polls & Surveys

- MPs and admins can create polls with multiple options and a defined duration.
- Constituents can vote once per poll; results are displayed in real time (or after closing).
- Polls can be attached to specific bills or standalone.
- Users can unsubscribe from poll notifications (see Background Processes).

5. Document Repository

- Upload and store official documents (PDF, DOCX, etc.) related to bills, committee reports, and public records.
- Searchable by title, tags, date, and document type.
- Access control: some documents may be restricted to MPs/Admins.

6. MP Profiles & Admin Panel

- Each MP has a public profile showing:
 - Contact information, constituency, party affiliation.
 - Voting history on bills.
 - Committee memberships.
 - Recent activity (bills proposed, forum posts).
- Admins can manage user accounts, roles, and content moderation.
- Dashboard with analytics: number of bills passed, active users, poll participation, etc.

7. Notifications & Alerts

- In-app notification center (bell icon) showing recent activities.
- Email notifications for important events (welcome, password reset, bill/poll alerts).
- Users can opt out of non-essential email notifications.

8. Search & Discovery

- Global search across bills, documents, forum posts, and users.
 - Advanced filtering and sorting options.
 - Location-based features (e.g., MP by constituency) if needed.
-



UI Requirements

Design Principles

- **Responsive:** mobile-first, works across devices.
- **Accessible:** comply with WCAG 2.1 AA.
- **Intuitive:** clear navigation, consistent layout, minimal learning curve.

Styling & Components

- Use **Bootstrap 5** (CSS framework) for layout, forms, modals, and utilities.
- Customize Bootstrap theme via SCSS variables to match brand (if any).
- Use **React Bootstrap** for React-friendly components (optional but recommended).

Key Interface Pages

- **Home / Dashboard:** latest announcements, trending bills, quick links.
- **Bill List & Detail:** table/cards with filters; detail view with timeline and comments.
- **Forum Index & Thread:** category list, thread view with pagination.
- **Poll Listing & Voting:** current polls, vote button, results display.
- **Document Library:** searchable grid of documents.
- **MP Profile:** public profile with voting history.
- **Admin Dashboard:** data tables, CRUD modals, charts (use Chart.js or Recharts).
- **User Profile:** edit personal info, manage notification preferences.

State Management (Frontend)

- Use **Zustand** stores for:
 - Authentication state (user, tokens, roles).
 - Bills (list, filters, selected bill).
 - Notifications (unread count, toast messages).

- UI state (modals, loading indicators).
- Persist auth tokens to `localStorage` with Zustand middleware.

Frontend Libraries

- **React Router** for routing.
- **Axios** for HTTP requests with interceptors to attach tokens and handle 401.
- **React Hook Form + Zod** for form validation.
- **Bootstrap 5** and **React Bootstrap** for styling.
- **date-fns** for date formatting.
- **Socket.IO client** if live chat is implemented.

⚙️ Backend Requirements

Architecture

- **FastAPI** framework (Python 3.9+).
- RESTful API with automatic OpenAPI documentation (Swagger UI / ReDoc).
- Modular structure using APIRouter.
- Stateless JWT authentication.

Project Structure (Backend)

```
backend/
├── app/
│   ├── main.py           # FastAPI app instance
│   ├── core/             # config, security, dependencies
│   ├── models/           # SQLAlchemy models
│   ├── schemas/          # Pydantic schemas
│   ├── api/              # route modules
│   │   ├── auth.py
│   │   ├── users.py
│   │   ├── bills.py
│   │   ├── forums.py
│   │   ├── polls.py
│   │   ├── documents.py
│   │   └── notifications.py
│   ├── services/         # business logic
│   ├── background/       # background tasks and workers
│   └── utils/
└── tests/
```

```
└─ requirements.txt
└─ .env.example
```

API Endpoints (Summary)

Method	Endpoint	Description	Access
POST	/api/auth/register	Register new user (send welcome email)	Public
POST	/api/auth/login	Login, returns JWT tokens	Public
POST	/api/auth/refresh	Refresh access token	Public
POST	/api/auth/password-reset/request	Send password reset email	Public
POST	/api/auth/password-reset/confirm	Reset password with token	Public
DELETE	/api/users/me	Delete own account (send goodbye email)	Authenticated
GET	/api/bills	List bills with filters	Public
POST	/api/bills	Create new bill	MP, Admin
PATCH	/api/bills/{id}	Update bill status/details	MP, Admin
GET	/api/forums	List forum categories/threads	Public
POST	/api/forums/threads	Create new thread	Authenticated
GET	/api/polls	List active polls	Public
POST	/api/polls/{id}/vote	Submit a vote	Authenticated
GET	/api/documents	List documents	Public
POST	/api/documents	Upload document	MP, Admin
GET	/api/users/{id}/profile	Get MP public profile	Public

Method	Endpoint	Description	Access
GET	/api/admin/stats	Dashboard analytics	Admin

Background Processes

- **Welcome Email:** On successful user registration, a background task sends a welcome email with a verification link (if applicable).
- **Goodbye Email:** When a user deletes their account, a background task sends a farewell email confirming deletion.
- **Bill/Poll Notifications:** When a new bill or poll is created, an email notification is sent to all users who have **not unsubscribed** from such alerts. This is processed asynchronously to avoid blocking the API.
- **Password Reset Request:** A background task sends a password reset email containing a secure token link.
- **Password Reset Confirmation:** After a successful password reset, a confirmation email is sent.
- **Implementation:** Use **FastAPI BackgroundTasks** for simple tasks, or a more robust queue (Celery + Redis) for scalability. For an MVP, BackgroundTasks is sufficient; for production, consider Celery.

Session Management

- **JWT Tokens:** Access token (short-lived, e.g., 15 minutes) and refresh token (long-lived, e.g., 7 days).
- **Token Storage:** Client stores tokens in `localStorage` (or secure, HTTP-only cookies if preferred). Refresh token can be rotated.
- **Token Revocation:** Maintain a denylist of revoked refresh tokens in the database (or Redis) to invalidate sessions immediately.
- **Session Expiry:** Access token expiration is handled by the frontend intercepting 401 responses and using the refresh token to obtain a new access token.
- **Logout:** Client deletes tokens; optionally invalidate the refresh token on the server.
- **Password Reset Tokens:** Short-lived (e.g., 1 hour), single-use, stored hashed in the database.
- **Security Headers:** Set appropriate CORS, X-Frame-Options, and other security headers.

Security

- Hash passwords with `bcrypt` or `argon2` (via `passlib`).
- Validate all inputs with Pydantic schemas.
- Protect against SQL injection using SQLAlchemy ORM.
- Sanitize user-generated content to prevent XSS.

- Rate limiting (e.g., `slowapi`) on authentication and public endpoints.
- File upload validation (size, MIME type) and store outside web root.
- Role-based permissions enforced via FastAPI dependencies.

Database

- **MariaDB** with SQLAlchemy ORM and `pymysql` driver.
- Use Alembic for migrations.
- See Database Requirements below.

Database Requirements

Schema Outline (Tables)

1. users

- `id` (PK), `email` (unique), `password_hash`, `full_name`, `role` (enum: constituent, mp, admin), `constituency_id` (FK, nullable), `party`, `is_active`, `email_verified`, `notification_opt_out` (boolean), `created_at`, `updated_at`.

2. constituencies

- `id` (PK), `name`, `region`.

3. bills

- `id` (PK), `title`, `description`, `sponsor_mp_id` (FK → users), `status` (enum), `category`, `introduced_date`, `updated_at`.

4. bill_documents

- `id` (PK), `bill_id` (FK), `document_id` (FK) – links bills to documents.

5. bill_actions

- `id` (PK), `bill_id` (FK), `action_type` (e.g., introduced, committee review), `description`, `action_date`, `performed_by` (FK → users).

6. forums

- `id` (PK), `title`, `description`, `category`, `created_by` (FK), `created_at`.

7. forum_threads

- `id` (PK), `forum_id` (FK), `title`, `created_by` (FK), `created_at`, `is_pinned`, `is_locked`.

8. forum_posts

- `id` (PK), `thread_id` (FK), `user_id` (FK), `content`, `created_at`, `updated_at`.

9. polls

- `id` (PK), `question`, `created_by` (FK), `start_date`, `end_date`, `is_active`, `bill_id` (FK, nullable).

10. poll_options

- `id` (PK), `poll_id` (FK), `option_text`.

11. poll_votes

- `id` (PK), `poll_id` (FK), `option_id` (FK), `user_id` (FK), `voted_at`. Unique (`poll_id`, `user_id`).

12. documents

- `id` (PK), `title`, `description`, `file_path`, `document_type` (enum), `uploaded_by` (FK), `uploaded_at`, `is_public`.

13. mp_profiles

- `user_id` (PK, FK → users), `bio`, `committee_memberships` (text or separate table), `voting_record` (link to `bill_votes` table).
- Alternatively, store committee memberships in a separate join table.

14. bill_votes (for MP voting records)

- `id` (PK), `bill_id` (FK), `mp_user_id` (FK), `vote` (yes/no/abstain), `vote_date`.

15. notifications

- `id` (PK), `user_id` (FK), `message`, `link`, `is_read`, `created_at`.

16. refresh_tokens

- `id` (PK), `user_id` (FK), `token_hash`, `expires_at`, `revoked`, `created_at`.

17. password_reset_tokens

- `id` (PK), `user_id` (FK), `token_hash`, `expires_at`, `used`.

Indexing & Constraints

- Indexes on foreign keys and frequently queried columns (`bills.status`, `forum_posts.thread_id`, `users.email`).
- Unique constraints on `users.email`, `poll_votes` (`poll_id`, `user_id`), etc.
- Use `ON DELETE CASCADE` where appropriate (e.g., deleting a bill removes its actions/documents).

Additional Considerations

Development & Deployment

- **Environment Configuration:** Use `.env` files and `pydantic-settings` for FastAPI; `Vite` env variables for React.
- **Containerization:** Provide Dockerfiles and `docker-compose.yml` for frontend, backend, and MariaDB.
- **CI/CD:** GitHub Actions for running tests and deploying.
- **Testing:**
 - Backend: `pytest` with FastAPI `TestClient`, mock background tasks.
 - Frontend: `Vitest` + `React Testing Library`.
- **API Documentation:** FastAPI auto-generates Swagger UI at `/docs`.

Background Task Queue

- For production, integrate **Celery** with **Redis** as broker to handle email sending and heavy tasks asynchronously.
- Configure retries and error logging.

Future Enhancements

- Real-time live chat via WebSockets (use Socket.IO or FastAPI WebSockets).
- AI analysis of citizen input (sentiment analysis, bill summarization).
- Mobile app (React Native or PWA).
- Multi-language support.
- Social login (Google, Facebook).

Tech Stack Summary

Layer	Technology
Frontend	React, Zustand, React Router, Axios, Bootstrap 5, React Bootstrap
Backend	FastAPI, SQLAlchemy, Pydantic, JWT, passlib, Celery (optional)
Database	MariaDB
DevOps	Docker, GitHub Actions, Nginx (reverse proxy)

Project Structure (Recommended)

```
citizen-connect-360/
├── frontend/                                # React app
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── stores/                        # Zustand stores
│   │   ├── services/                     # API calls
│   │   ├── utils/
│   │   ├── App.jsx
│   └── main.jsx
```

```
|   └─ package.json
└─ backend/                                # FastAPI app
    └─ app/
        ├── main.py
        ├── core/
        ├── models/
        ├── schemas/
        ├── api/
        ├── services/
        └─ background/
    ├── tests/
    ├── requirements.txt
    └─ .env.example
└─ docker-compose.yml
└─ README.md
```

Background Processes (Detailed)

These are essential asynchronous operations triggered by user actions or system events. For a scalable implementation, use a task queue (Celery) but FastAPI's `BackgroundTasks` can be used for simple, lightweight tasks.

Trigger	Task	Implementation
New user registration	Send welcome email with verification link	BackgroundTasks / Celery
Account deletion	Send goodbye email	BackgroundTasks / Celery
New bill or poll created	Notify all users (who haven't unsubscribed) via email	Celery (batch job)
Password reset requested	Send password reset email with token	BackgroundTasks / Celery
Password reset successful	Send confirmation email	BackgroundTasks / Celery

User Notification Preferences: Users can opt out of bill/poll notifications via a boolean field (`notification_opt_out`) in the user profile. The background process checks this flag before sending.



Session Management Specifications

- **Token Generation:** On login, the backend issues an **access token** (JWT, exp 15 min) and a **refresh token** (opaque or JWT, exp 7 days). The refresh token is stored hashed in the `refresh_tokens` table.
- **Token Storage:** The frontend stores tokens in `localStorage` (or secure, HTTP-only cookies if the frontend and backend are on the same domain, but cross-domain requires `localStorage` with careful XSS protection).
- **Refresh Flow:** When the access token expires, the frontend calls `/api/auth/refresh` with the refresh token. The server validates it, checks it's not revoked, and issues a new pair. Optionally rotate the refresh token.
- **Revocation:** On logout, the refresh token is revoked (marked as revoked or deleted). Password reset tokens are single-use.
- **Security:** Tokens are signed with a strong secret; use HTTPS in production. Consider using `httpOnly` cookies for refresh token to mitigate XSS.